

ASSIGNMENT-11.3

Name: G.Mythili

Ht.no: 2303A51283

Batch: 05

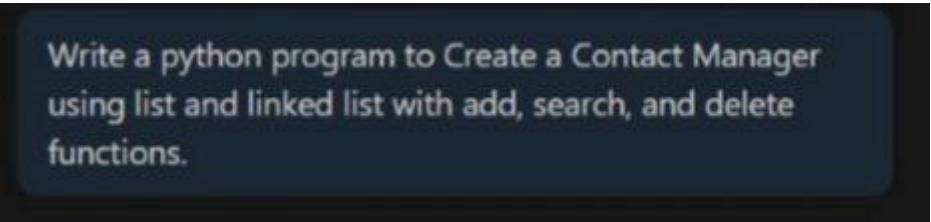
Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Prompt:

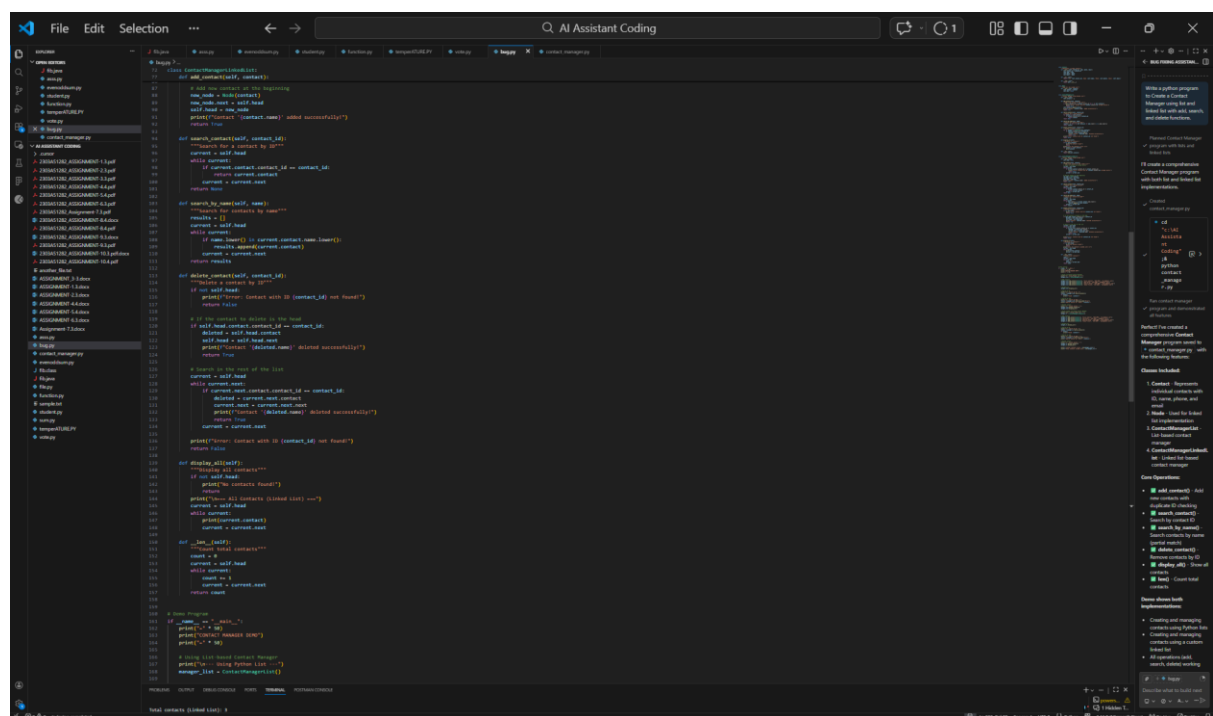
Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.



Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.

```

1 class Contact:
2     """Represents a single contact"""
3     def __init__(self, contact_id, name, phone, email):
4         self.contact_id = contact_id
5         self.name = name
6         self.phone = phone
7         self.email = email
8
9     def __str__(self):
10         return f'({self.contact_id}, {name} {self.phone}, {self.email})'
11
12     def __repr__(self):
13         return self.__str__()
14
15     def __eq__(self, other):
16         """Make for the linked list"""
17         return self.name == other.name
18
19     def __lt__(self, other):
20         """Make for the linked list"""
21         return self.name < other.name
22
23     def __gt__(self, other):
24         """Make for the linked list"""
25         return self.name > other.name
26
27 class ContactManager:
28     """Contact Manager using Python class"""
29     def __init__(self):
30         self.contacts = []
31
32     def add_contact(self, contact):
33         """Add a new contact to the list"""
34         if not contact_id or not contact.name or not contact.phone or not contact.email:
35             print("Error: Contact with ID, name, phone, email already exists")
36             return False
37         self.contacts.append(contact)
38         print(f"Contact {contact.name} added successfully")
39         return True
40
41     def delete_contact(self, contact_id):
42         """Delete a contact by ID"""
43         for i, contact in enumerate(self.contacts):
44             if contact.contact_id == contact_id:
45                 del self.contacts[i]
46                 print(f"Contact {contact.name} deleted successfully")
47                 return True
48         print("Error: Contact with ID (contact_id) not found")
49         return False
50
51     def display_all(self):
52         """Display all contacts"""
53         if not self.contacts:
54             print("No contacts found")
55             return
56         print("\nAll contacts (list) ---")
57         for contact in self.contacts:
58             print(contact)
59
60     def __len__(self):
61         return len(self.contacts)
62
63 class ContactManagerUsingLinked:
64     """Contact Manager using linked list"""
65     def __init__(self):
66         self.head = None
67
68     def add_contact(self, contact):
69         """Add a new contact to the linked list"""
70         if not contact_id or not contact.name or not contact.phone or not contact.email:
71             print("Error: Contact with ID, name, phone, email already exists")
72             return False
73         current = self.head
74         while current:
75             if current.contact_id == contact_id:
76                 print("Error: Contact with ID (contact_id) already exists")
77                 return False
78             current = current.next
79         current = contact
80         current.next = None
81
82     def delete_contact(self, contact_id):
83         """Delete a contact by ID"""
84         if not self.head:
85             print("No contacts found")
86             return
87         current = self.head
88         while current:
89             if current.contact_id == contact_id:
90                 print(f"Contact {current.name} deleted successfully")
91                 current = current.next
92                 return True
93             current = current.next
94         print("Error: Contact with ID (contact_id) not found")
95         return False
96
97     def display_all(self):
98         """Display all contacts"""
99         if not self.head:
100             print("No contacts found")
101             return
102         current = self.head
103         while current:
104             print(current)
105             current = current.next
106
107     def __len__(self):
108         """Return the number of contacts"""
109         if not self.head:
110             return 0
111         current = self.head
112         count = 0
113         while current:
114             count += 1
115             current = current.next
116         return count
117
118 if __name__ == '__main__':
119     manager = ContactManager()
120     manager.add_contact(Contact(1, "John Doe", "123-456-7890", "john.doe@example.com"))
121     manager.add_contact(Contact(2, "Jane Smith", "987-654-3210", "jane.smith@example.com"))
122     manager.add_contact(Contact(3, "Bob Johnson", "555-111-2222", "bob.johnson@example.com"))
123     manager.add_contact(Contact(4, "Alice Brown", "444-333-2222", "alice.brown@example.com"))
124     manager.add_contact(Contact(5, "Charlie White", "333-222-1111", "charlie.white@example.com"))
125     manager.add_contact(Contact(6, "Diana Green", "222-111-0000", "diana.green@example.com"))
126     manager.add_contact(Contact(7, "Ethan Black", "111-000-9999", "ethan.black@example.com"))
127     manager.add_contact(Contact(8, "Fiona Grey", "000-999-8888", "fiona.grey@example.com"))
128     manager.add_contact(Contact(9, "George Blue", "999-888-7777", "george.blue@example.com"))
129     manager.add_contact(Contact(10, "Helen Pink", "888-777-6666", "helen.pink@example.com"))
130     manager.add_contact(Contact(11, "Ivan Yellow", "777-666-5555", "ivan.yellow@example.com"))
131     manager.add_contact(Contact(12, "Julia Purple", "666-555-4444", "julia.purple@example.com"))
132     manager.add_contact(Contact(13, "Kevin Orange", "555-444-3333", "kevin.orange@example.com"))
133     manager.add_contact(Contact(14, "Laura Silver", "444-333-2222", "laura.silver@example.com"))
134     manager.add_contact(Contact(15, "Miguel Gold", "333-222-1111", "miguel.gold@example.com"))
135     manager.add_contact(Contact(16, "Nora Bronze", "222-111-0000", "nora.bronze@example.com"))
136     manager.add_contact(Contact(17, "Oscar Copper", "111-000-9999", "oscar.copper@example.com"))
137     manager.add_contact(Contact(18, "Pamela Iron", "000-999-8888", "pamela.iron@example.com"))
138     manager.add_contact(Contact(19, "Quinn Steel", "999-888-7777", "quinn.steel@example.com"))
139     manager.add_contact(Contact(20, "Ramon Tin", "888-777-6666", "ramon.tin@example.com"))
140     manager.add_contact(Contact(21, "Sara Nickel", "777-666-5555", "sara.nickel@example.com"))
141     manager.add_contact(Contact(22, "Tina Zinc", "666-555-4444", "tina.zinc@example.com"))
142     manager.add_contact(Contact(23, "Uma Lead", "555-444-3333", "uma.lead@example.com"))
143     manager.add_contact(Contact(24, "Victor Platinum", "444-333-2222", "victor.platinum@example.com"))
144     manager.add_contact(Contact(25, "Wendy Silver", "333-222-1111", "wendy.silver@example.com"))
145     manager.add_contact(Contact(26, "Xavier Gold", "222-111-0000", "xavier.gold@example.com"))
146     manager.add_contact(Contact(27, "Yara Bronze", "111-000-9999", "yara.bronze@example.com"))
147     manager.add_contact(Contact(28, "Zoe Copper", "000-999-8888", "zoe.copper@example.com"))
148     manager.add_contact(Contact(29, "Adam Iron", "999-888-7777", "adam.iron@example.com"))
149     manager.add_contact(Contact(30, "Eve Steel", "888-777-6666", "eve.steel@example.com"))
150     manager.add_contact(Contact(31, "Frank Tin", "777-666-5555", "frank.tin@example.com"))
151     manager.add_contact(Contact(32, "Grace Nickel", "666-555-4444", "grace.nickel@example.com"))
152     manager.add_contact(Contact(33, "Harry Zinc", "555-444-3333", "harry.zinc@example.com"))
153     manager.add_contact(Contact(34, "Ivy Lead", "444-333-2222", "ivy.lead@example.com"))
154     manager.add_contact(Contact(35, "Jack Platinum", "333-222-1111", "jack.platinum@example.com"))
155     manager.add_contact(Contact(36, "Karen Silver", "222-111-0000", "karen.silver@example.com"))
156     manager.add_contact(Contact(37, "Leo Gold", "111-000-9999", "leo.gold@example.com"))
157     manager.add_contact(Contact(38, "Mia Bronze", "000-999-8888", "mia.bronze@example.com"))
158     manager.add_contact(Contact(39, "Noah Copper", "999-888-7777", "noah.copper@example.com"))
159     manager.add_contact(Contact(40, "Olivia Iron", "888-777-6666", "olivia.iron@example.com"))
160     manager.add_contact(Contact(41, "Peter Steel", "777-666-5555", "peter.steel@example.com"))
161     manager.add_contact(Contact(42, "Quinn Tin", "666-555-4444", "quinn.tin@example.com"))
162     manager.add_contact(Contact(43, "Ramon Nickel", "555-444-3333", "ramon.nickel@example.com"))
163     manager.add_contact(Contact(44, "Sara Zinc", "444-333-2222", "sara.zinc@example.com"))
164     manager.add_contact(Contact(45, "Tina Lead", "333-222-1111", "tina.lead@example.com"))
165     manager.add_contact(Contact(46, "Uma Platinum", "222-111-0000", "uma.platinum@example.com"))
166     manager.add_contact(Contact(47, "Victor Silver", "111-000-9999", "victor.silver@example.com"))
167     manager.add_contact(Contact(48, "Wendy Gold", "000-999-8888", "wendy.gold@example.com"))
168     manager.add_contact(Contact(49, "Xavier Bronze", "999-888-7777", "xavier.bronze@example.com"))
169     manager.add_contact(Contact(50, "Yara Copper", "888-777-6666", "yara.copper@example.com"))
170     manager.add_contact(Contact(51, "Zoe Iron", "777-666-5555", "zoe.iron@example.com"))
171     manager.add_contact(Contact(52, "Adam Steel", "666-555-4444", "adam.steel@example.com"))
172     manager.add_contact(Contact(53, "Eve Tin", "555-444-3333", "eve.tin@example.com"))
173     manager.add_contact(Contact(54, "Frank Nickel", "444-333-2222", "frank.nickel@example.com"))
174     manager.add_contact(Contact(55, "Grace Zinc", "333-222-1111", "grace.zinc@example.com"))
175     manager.add_contact(Contact(56, "Harry Lead", "222-111-0000", "harry.lead@example.com"))
176     manager.add_contact(Contact(57, "Ivy Platinum", "111-000-9999", "ivy.platinum@example.com"))
177     manager.add_contact(Contact(58, "Jack Silver", "000-999-8888", "jack.silver@example.com"))
178     manager.add_contact(Contact(59, "Karen Gold", "999-888-7777", "karen.gold@example.com"))
179     manager.add_contact(Contact(60, "Leo Bronze", "888-777-6666", "leo.bronze@example.com"))
180     manager.add_contact(Contact(61, "Mia Copper", "777-666-5555", "mia.copper@example.com"))
181     manager.add_contact(Contact(62, "Noah Iron", "666-555-4444", "noah.iron@example.com"))
182     manager.add_contact(Contact(63, "Olivia Steel", "555-444-3333", "olivia.steel@example.com"))
183     manager.add_contact(Contact(64, "Peter Tin", "444-333-2222", "peter.tin@example.com"))
184     manager.add_contact(Contact(65, "Quinn Nickel", "333-222-1111", "quinn.nickel@example.com"))
185     manager.add_contact(Contact(66, "Ramon Zinc", "222-111-0000", "ramon.zinc@example.com"))
186     manager.add_contact(Contact(67, "Sara Lead", "111-000-9999", "sara.lead@example.com"))
187     manager.add_contact(Contact(68, "Tina Platinum", "000-999-8888", "tina.platinum@example.com"))
188     manager.add_contact(Contact(69, "Uma Silver", "999-888-7777", "uma.silver@example.com"))
189     manager.add_contact(Contact(70, "Victor Gold", "888-777-6666", "victor.gold@example.com"))
190     manager.add_contact(Contact(71, "Wendy Bronze", "777-666-5555", "wendy.bronze@example.com"))
191     manager.add_contact(Contact(72, "Xavier Copper", "666-555-4444", "xavier.copper@example.com"))
192     manager.add_contact(Contact(73, "Yara Iron", "555-444-3333
```





- In an array, adding at the end is fast, but inserting in the middle is slow because elements must shift.
- In a linked list, insertion is fast because no shifting is needed.
- Searching takes the same time in both (you must check each element).
- Deleting in an array is slower due to shifting elements.
- Linked list is better for frequent insertions and deletions.

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Prompt:

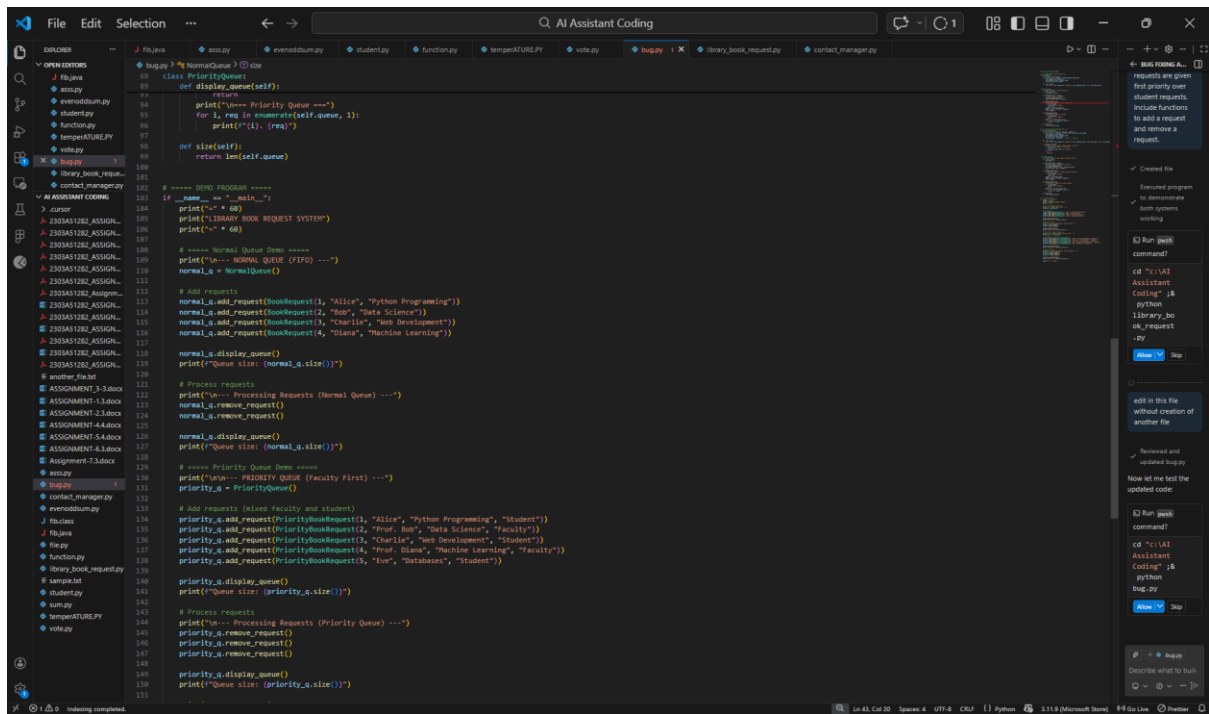
Write a Python program for a library book request system. First, make a normal queue where requests are handled in the order they come. Then, make another version where faculty requests are given first priority over student requests. Include functions to add a request and remove a request.

Write a Python program for a library book request system.
First, make a normal queue where requests are handled in the order they come.
Then, make another version where faculty requests are given first priority over student requests.
Include functions to add a request and remove a request.

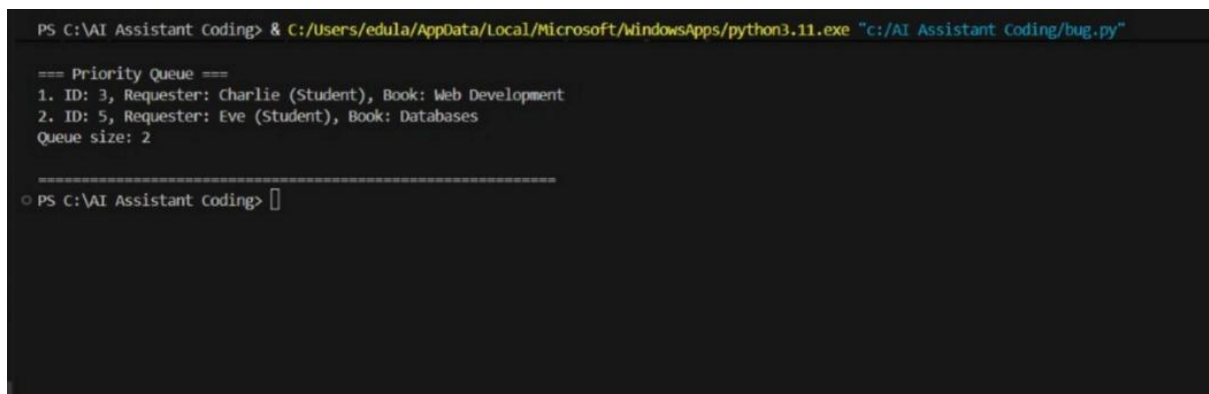
Code:

```
File Edit Selection ... Q AI Assistant Coding
buggy > % NormalQueue > remove_request
1 from collections import deque
2
3 # ===== NORMAL QUEUE IMPLEMENTATION =====
4 class NormalQueue:
5     """represents a book request"""
6     def __init__(self, request_id, requester_name, book_title):
7         self.request_id = request_id
8         self.requester_name = requester_name
9         self.book_title = book_title
10
11     def __str__(self):
12         return f"ID: {self.request_id}, Requester: {self.requester_name}, Book: {self.book_title}"
13
14
15 class NormalQueue:
16     """Simple FIFO queue for book requests"""
17     def __init__(self):
18         self.queue = deque()
19
20     def add_request(self, request):
21         """add a request to the queue"""
22         self.queue.append(request)
23         print("Request added: (request)")
24
25     def remove_request(self):
26         """remove and return the first request"""
27         if not self.queue:
28             print("Queue is empty!")
29             return None
30         request = self.queue.popleft()
31         print("Request processed: (request)")
32         return request
33
34     def display_queue(self):
35         """display all requests in queue"""
36         if not self.queue:
37             print("Queue is empty!")
38             return
39         print("===== Normal Queue =====")
40         for i, req in enumerate(self.queue, 1):
41             print(f"({i}). {req}")
42
43     def size(self):
44         return len(self.queue)
45
46 # ===== PRIORITY QUEUE IMPLEMENTATION =====
47 class PriorityBookRequest:
48     """book request with priority (Faculty > Student)"""
49     def __init__(self, request_id, requester_name, book_title, user_type):
50         self.request_id = request_id
51         self.requester_name = requester_name
52         self.book_title = book_title
53         self.user_type = user_type # "Faculty" or "Student"
54
55     def __str__(self):
56         return f"ID: {self.request_id}, Requester: {self.requester_name} ({self.user_type}), Book: {self.book_title}"
57
58     def get_priority(self):
59         """return priority (lower number = higher priority)"""
60         if self.user_type.lower() == "faculty":
61             return 1
62         else:
63             return 2
```

```
File Edit Selection ... Q AI Assistant Coding
buggy > % NormalQueue > remove_request
64
65 class PriorityBookRequest:
66     """book request with priority (Faculty > Student)"""
67     def __init__(self, request_id, requester_name, book_title, user_type):
68         self.request_id = request_id
69         self.requester_name = requester_name
70         self.book_title = book_title
71         self.user_type = user_type # "Faculty" or "Student"
72
73     def __str__(self):
74         return f"ID: {self.request_id}, Requester: {self.requester_name} ({self.user_type}), Book: {self.book_title}"
75
76     def get_priority(self):
77         """return priority (lower number = higher priority)"""
78         if self.user_type.lower() == "faculty":
79             return 1
80         else:
81             return 2
82
83     def add_request(self, request):
84         """add a request with priority sorting"""
85         self.queue.append(request)
86         self.queue.sort(key=lambda x: x.get_priority())
87         print("Request added: (request)")
88
89     def remove_request(self):
90         """remove and return the highest priority request"""
91         if not self.queue:
92             print("Queue is empty!")
93             return None
94         request = self.queue.pop(0)
95         print("Request processed: (request)")
96         return request
97
98     def display_queue(self):
99         """display all requests in priority order"""
100         if not self.queue:
101             print("Queue is empty!")
102             return
103         print("===== Priority Queue =====")
104         for i, req in enumerate(self.queue, 1):
105             print(f"({i}). {req}")
106
107     def size(self):
108         return len(self.queue)
109
110 # ===== DEMO PROGRAM =====
111 if __name__ == "__main__":
112     print("===== LIBRARY BOOK REQUEST SYSTEM =====")
113     # Create Normal Queue Demo
114     normal_q = NormalQueue()
115     # Add requests
116     normal_q.add_request(BookRequest(1, "Alice", "Python Programming"))
117     normal_q.add_request(BookRequest(2, "Bob", "Data Science"))
118     normal_q.add_request(BookRequest(3, "Charlie", "Web Development"))
119     normal_q.add_request(BookRequest(4, "Diana", "Machine Learning"))
120     normal_q.display_queue()
121     print(f"Queue size: {normal_q.size()}")
122
123     # Process requests
124     print("===== Processing Requests (Normal Queue) =====")
125     normal_q.remove_request()
126     normal_q.remove_request()
127     normal_q.display_queue()
128     print(f"Queue size: {normal_q.size()}")
129
130     # Create Priority Queue Demo
131     priority_q = PriorityBookRequest()
132     # Add requests
133     priority_q.add_request(BookRequest(1, "Alice", "Python Programming", "Student"))
134     priority_q.add_request(BookRequest(2, "Bob", "Data Science", "Faculty"))
135     priority_q.add_request(BookRequest(3, "Charlie", "Web Development", "Student"))
136     priority_q.add_request(BookRequest(4, "Diana", "Machine Learning", "Faculty"))
137     priority_q.display_queue()
138     print(f"Queue size: {priority_q.size()}")
139
140     # Process requests
141     print("===== Processing Requests (Priority Queue) =====")
142     priority_q.remove_request()
143     priority_q.remove_request()
144     priority_q.display_queue()
145     print(f"Queue size: {priority_q.size()}")
```



Output:



Explanation:

- Queue (FIFO) → First request comes, first served.(If a student requests first, they get the book first.)
- Priority Queue → Faculty requests are served before students, even if they come later.
- enqueue() → Adds a request to the system.

- `dequeue()` → Removes and processes the next request.

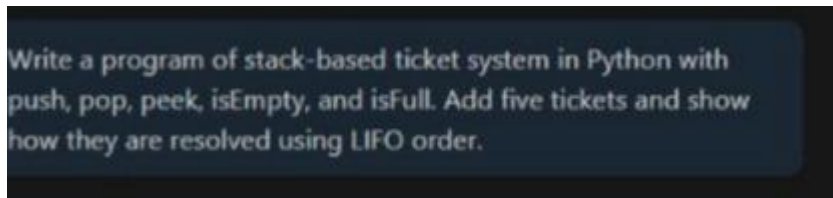
Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Prompt:

Write a program of stack-based ticket system in Python with `push`, `pop`, `peek`, `isEmpty`, and `isFull`. Add five tickets and show how they are resolved using LIFO order.



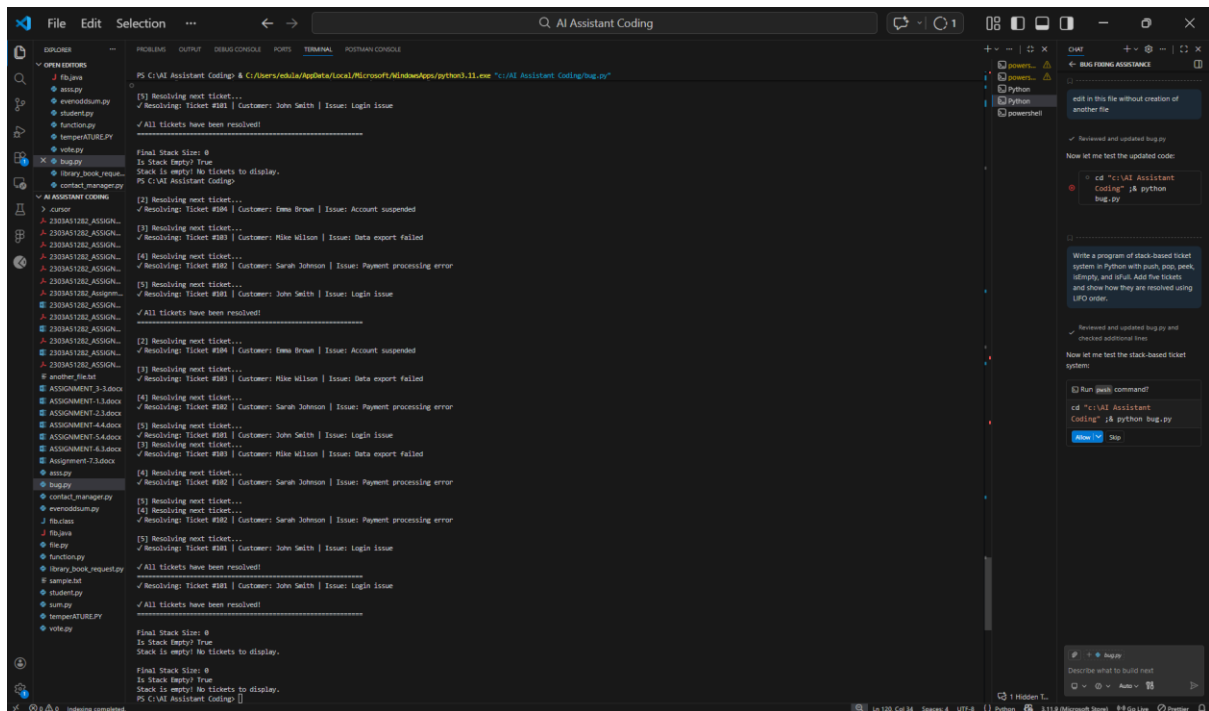
Write a program of stack-based ticket system in Python with `push`, `pop`, `peek`, `isEmpty`, and `isFull`. Add five tickets and show how they are resolved using LIFO order.

Code:

```
1 # ===== STACK-BASED TICKET SYSTEM =====
2 class Ticket:
3     """Represents a support ticket"""
4     def __init__(self, ticket_id, customer_name, issue):
5         self.ticket_id = ticket_id
6         self.customer_name = customer_name
7         self.issue = issue
8
9     def __str__(self):
10         return f"Ticket {self.ticket_id} | Customer: {self.customer_name} | Issue: {self.issue}"
11
12
13 class TicketStack:
14     """Stack-based ticket management system (LIFO - Last In, First Out)"""
15     def __init__(self, max_size=10):
16         self.stack = []
17         self.max_size = max_size
18
19     def push(self, ticket):
20         """Add a ticket to the stack (top of stack)"""
21         if self.is_full():
22             print(f"Error: Stack is full! Cannot add ticket {ticket.ticket_id}")
23             return False
24         self.stack.append(ticket)
25         print(f"/> Resolving {ticket}")
26         return True
27
28     def pop(self):
29         """Remove and return the ticket from the top of the stack"""
30         if self.is_empty():
31             print(f"Error: Stack is empty! No tickets to resolve.")
32             return None
33         ticket = self.stack.pop()
34         print(f"/> Resolving {ticket}")
35         return ticket
36
37     def peek(self):
38         """Return the top ticket without removing it"""
39         if self.is_empty():
40             print(f"Error: Stack is empty!")
41             return None
42         return self.stack[-1]
43
44     def is_empty(self):
45         """Check if the stack is empty"""
46         return len(self.stack) == 0
47
48     def is_full(self):
49         """Check if the stack is full"""
50         return len(self.stack) >= self.max_size
51
52     def size(self):
53         """Return the number of tickets in the stack"""
54         return len(self.stack)
55
56     def display_stack(self):
57         """Display all tickets in the stack (top to bottom)"""
```

```
58     def resolve_all(self):
59         """Resolve all tickets in LIFO order"""
60         print("\n" + "-" * 60)
61         print(f"Resolving ALL TICKETS (LIFO - Last In, First Out)")
62         print("\n" + "-" * 60)
63         count = 1
64         while not self.is_empty():
65             print(f"Un{count} Resolving next ticket...")
66             self.pop()
67             count += 1
68         print("\n\n All tickets have been resolved!")
69         print("\n" + "-" * 60)
70
71
72 # ===== DEBUG PROGRAM =====
73 if __name__ == "__main__":
74     print("\n" + "-" * 60)
75     print("STACK-BASED TICKET SYSTEM")
76     print("\n" + "-" * 60)
77
78     # Create ticket stack with max size of 10
79     ticket_system = TicketStack(max_size=10)
80
81     # Add five tickets
82     print("\n--- ADDING TICKETS TO THE STACK ---\n")
83     ticket_system.push(ticket(100, "John Smith", "Login issue"))
84     ticket_system.push(ticket(101, "Sarah Johnson", "Payment processing error"))
85     ticket_system.push(ticket(102, "Mike Wilson", "Data export failed"))
86     ticket_system.push(ticket(103, "Emma Brown", "Account suspended"))
87     ticket_system.push(ticket(104, "David Lee", "Password reset not working"))
88
89     # Display current stack
90     ticket_system.display_stack()
91
92     # Show stack information
93     print("\nStack Size: (ticket_system.size())")
94     print(f"Is Stack Empty? (ticket_system.is_empty())")
95     print(f"Is Stack Full? (ticket_system.is_full())")
96
97     # Peek at the top ticket
98     print("\n--- PEAK AT TOP TICKET ---")
99     top_ticket = ticket_system.peak()
100     if top_ticket:
101         print(f"Top ticket (without removing): {top_ticket}")
102
103     # Resolve all tickets in LIFO order
104     ticket_system.resolve_all()
105
106     # Display final stack state
107     print("\nFinal Stack Size: (ticket_system.size())")
108     print(f"Is Stack Empty? (ticket_system.is_empty())")
109     ticket_system.display_stack()
```

Output:



Explanation:

The program uses a stack to manage help desk tickets.

A stack works in last in, first solved order.

When a new ticket is raised, it is added to the top.

When solving a ticket, the most recent one is handled first.

The program can also check if there are no tickets left or if the stack is full.

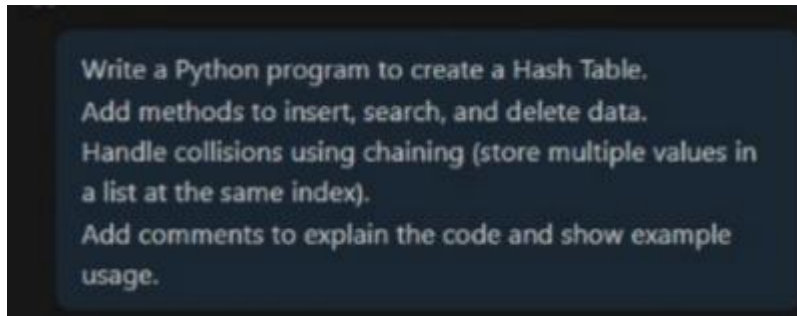
Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Prompt:

Write a Python program to create a Hash Table.
Add methods to insert, search, and delete data.
Handle collisions using chaining (store multiple values in a list at the same index).
Add comments to explain the code and show example usage.



Code:

```
File Edit Selection ... Q AI Assistant Coding
# Create Hash Table with Chaining Collision Handling
1 # Create key-value pair class for storing data
2 class KeyValuePair:
3     """Represents a key-value pair in the hash table"""
4     def __init__(self, key, value):
5         self.key = key
6         self.value = value
7
8     def __str__(self):
9         return f"({self.key} | {self.value})"
10
11 class HashTable:
12     """Hash Table Implementation using chaining to handle collisions.
13     Chaining stores multiple key-value pairs that hash to the same index in a linked list.
14     """
15     def __init__(self, size=10):
16         """Initialize the hash table
17         Args:
18             size (int): Number of buckets in the hash table
19         """
20         self.size = size
21         # Create an array of empty lists (chains) for collision handling
22         self.table = [[] for _ in range(size)]
23         self.total_items = 0
24
25     def _hash_function(self, key):
26         """Hash function to determine the index for a given key
27         Uses simple modulo operation: hash(key) % table_size
28         Args:
29             key: The key to hash
30         Returns:
31             int: Index in the hash table
32         """
33         # Convert key to string and sum ASCII values for different key types
34         key_str = str(key)
35         hash_value = sum(ord(char) for char in key_str)
36         return hash_value % self.size
37
38     def insert(self, key, value):
39         """Insert a key-value pair into the hash table
40         If key already exists, update its value
41         If collision occurs, add to the chain (list) at that index
42         Args:
43             key: The key to insert
44             value: The value associated with the key
45         Returns:
46             bool: True if insertion successful, False otherwise
47         """
48         # Get the hash index
49         index = self._hash_function(key)
50         chain = self.table[index]
51
52         # Check if key already exists in the chain and update if found
53         for i, pair in enumerate(chain):
54             if pair.key == key:
```

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(self.size)]

    def insert(self, key, value):
        # Hash the key to find the bucket
        bucket = self._hash(key)
        # Check if the key already exists in the bucket
        for i, pair in enumerate(self.table[bucket]):
            if pair[0] == key:
                # Key exists, update the value
                pair[1] = value
                return True
        # Key not found, add new key-value pair to the bucket
        self.table[bucket].append([key, value])
        return True

    def search(self, key):
        # Hash the key to find the bucket
        bucket = self._hash(key)
        # Search for the key in the bucket
        for i, pair in enumerate(self.table[bucket]):
            if pair[0] == key:
                return pair[1]
        # Key not found
        return None

    def delete(self, key):
        # Hash the key to find the bucket
        bucket = self._hash(key)
        # Search for the key in the bucket
        for i, pair in enumerate(self.table[bucket]):
            if pair[0] == key:
                # Key exists, delete it
                del self.table[bucket][i]
                return True
        # Key not found
        return False

    def _hash(self, key):
        # Simple hash function
        return hash(key) % self.size
```

```
def main():
    # Create a hash table with 10 buckets
    hash_table = HashTable(10)

    # Insert some data
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("email", "alice@example.com")
    hash_table.insert("phone", "123-456-7890")
    hash_table.insert("country", "USA")
    hash_table.insert("address", "123 Main St")

    # Display the hash table
    hash_table.display()

    # Search for data
    print("Searching for data...")
    hash_table.search("name")
    hash_table.search("age")
    hash_table.search("unknown_key") # Key that doesn't exist

    # Delete data
    print("Deleting data...")
    hash_table.delete("age")
    hash_table.delete("nonexistent") # Try to delete non-existent key

    # Display the hash table after deletion
    hash_table.display()

    # Get all items
    print("Getting all items...")
    all_items = hash_table.get_all_items()
    for item in all_items:
        print(item)
```

Output:

```
PS C:\VAI Assistant Coding\8 C:\Users\eduar\AppData\Local\Microsoft\WindowsApps\python.11.exe "C:\VAI Assistant Coding\bug.py"
Index 1: age: 30 -> city: New York -> department: IT
Index 2: name: Alice -> salary: 7000
Index 3: phone: 555-1234 -> country: USA
Index 4: [empty]
Total items in hash table: 8

--- SEARCHING FOR DATA ---
✓ Found: name = Alice (at index 2)
✓ Found: age = 30 (at index 1)
✗ Key 'unknown_key' not found in the hash table
2303AS1282_ASSIGN...
--- UPDATING DATA ---
✓ Updated: age = 31 (at index 1)
2303AS1282_ASSIGN...
HIGH TABLE CONTENTS
Index 0: email: alice@gmail.com
Index 1: age: 31 -> city: New York -> department: IT
Index 2: name: Alice -> salary: 7000
Index 3: phone: 555-1234 -> country: USA
Index 4: [empty]
Total items in hash table: 8

--- DELETING DATA ---
✓ Deleted: email = alice@gmail.com (from index 0)
✓ Deleted: city = New York (from index 1)
✗ Key 'nonexistent' not found in the hash table
2303AS1282_ASSIGN...
HIGH TABLE CONTENTS
Index 0: [empty]
Index 1: age: 31 -> department: IT
Index 2: name: Alice -> salary: 7000
Index 3: phone: 555-1234 -> country: USA
Index 4: [empty]
Total items in hash table: 6

--- ALL REPRIZING ITEMS ---
age: 31
dept: IT
name: Alice
salary: 7000
phone: 555-1234
country: USA
Total items: 6
Is empty: false
PS C:\VAI Assistant Coding\
```

Explanation:

- A Hash Table stores data using a key and value.
- A hash function decides where to store the data.
- Sometimes two keys go to the same place. This is called a collision.
- To solve collisions, we use chaining, meaning we store multiple items in a list at the same index.
- The program should allow adding, finding, and removing data correctly.

Task 5: Real-Time Application Challenge Scenario

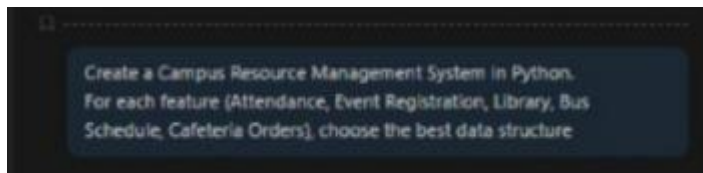
Design a Campus Resource Management System with the following features:

- Student Attendance Tracking

- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Prompt:

Create a Campus Resource Management System in Python. For each feature (Attendance, Event Registration, Library, Bus Schedule, Cafeteria Orders), choose the best data structure

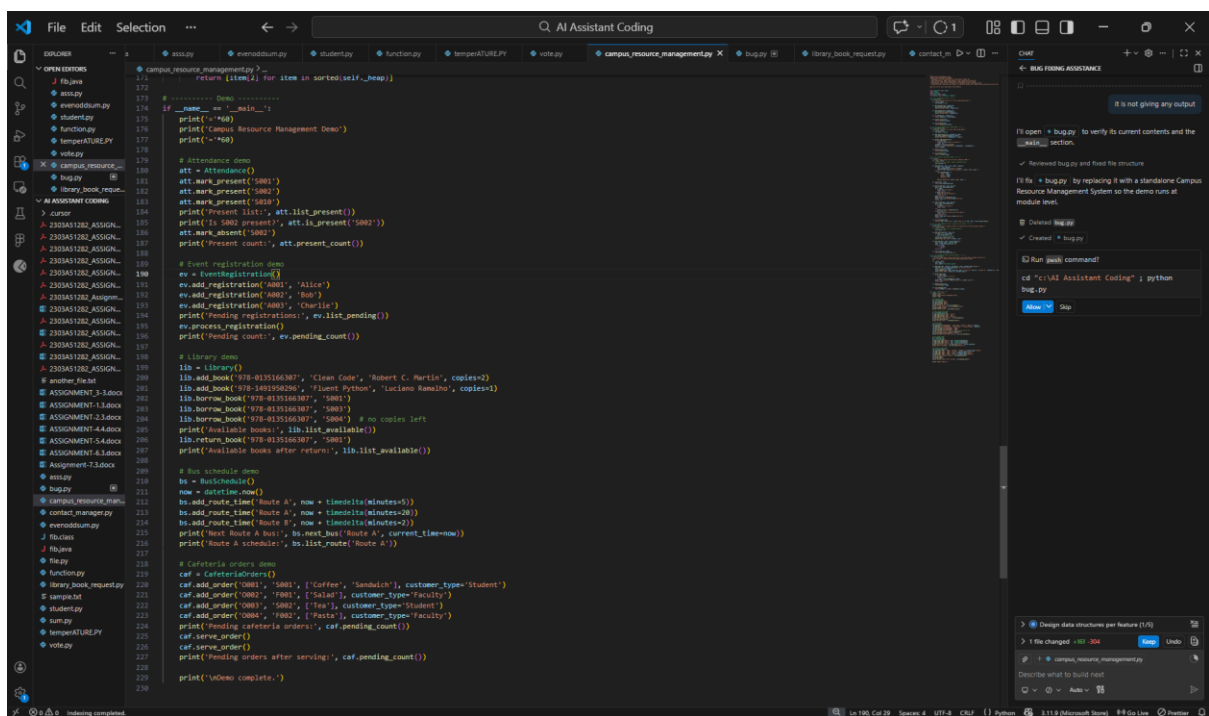


Code:

```

1  # Campus Resource Management System
2  # Features and chosen data structures:
3  # - Attendance: set (O(1) add/check) to track present student IDs
4  # - Event Registration: deque (FIFO queue) for fair registration processing
5  # - Library: dict (hash table) mapping ISBN -> book record for fast lookup
6  # - Bus Schedule: dict of route -> sorted list of departure times (list best sorted)
7  # - Cafeteria Orders: heap (priority queue) to prioritize faculty over students while preserving arrival order
8
9  Run this file to see a small demo of each feature.
10
11
12
13  from collections import deque
14  import heapq
15  import heapq
16  from bisect import bisect
17  from datetime import datetime, timedelta
18
19  # ----- Attendance (set) -----
20  class Attendance:
21      """Track attendance using a set for O(1) add/remove/check."""
22      def __init__(self):
23          self.present = set()
24
25      def mark_present(self, student_id):
26          self.present.add(student_id)
27          print(f"Marked present: {student_id}")
28
29      def mark_absent(self, student_id):
30          self.present.discard(student_id)
31          print(f"Marked absent: {student_id}")
32
33      def is_present(self, student_id):
34          return student_id in self.present
35
36      def present_count(self):
37          return len(self.present)
38
39      def list_present(self):
40          return sorted(self.present)
41
42  # ----- Event Registration (FIFO queue) -----
43  class EventRegistration:
44      """Register attendees in arrival order using deque."""
45      def __init__(self):
46          self.queue = deque()
47
48      def add_registration(self, attendee_id, name):
49          self.queue.append((attendee_id, name))
50          print(f"Registered: {attendee_id} - {name}")
51
52      def process_registration(self):
53          if not self.queue:
54              print("No registrations to process.")
55              return None
56          attendee = self.queue.popleft()
57          print(f"Processed registration: {attendee[0]} - {attendee[1]}")
58          return attendee
59
60      def pending_count(self):
61          return len(self.queue)

```



```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL POSTMAN CONSOLE

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
Is empty: False
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/campus_resource_management.py"
=====
Campus Resource Management Demo
=====
Marked present: 5001
Marked present: 5002
Marked present: 5010
Present list: ['5001', '5002', '5010']
Is 5002 present? True
Marked absent: 5002
Present count: 2
Registered: A001 - Alice
Registered: A002 - Bob
Registered: A003 - Charlie
Pending registrations: [('A001', 'Alice'), ('A002', 'Bob'), ('A003', 'Charlie')]
Processed registration: A001 - Alice
Pending count: 2
Added book: Clean Code (ISBN: 978-0135166307).
Added book: Fluent Python (ISBN: 978-1491958296).
5001 borrowed Clean Code
5003 borrowed Clean Code
No copies available.
Available books: [('978-0135166307', 'Clean Code', 0), ('978-1491958296', 'Fluent Python', 1)]
5001 returned Clean Code
Available books after return: [('978-0135166307', 'Clean Code', 1), ('978-1491958296', 'Fluent Python', 1)]
Added bus time for Route A: 2026-02-18 10:42:24.367227
Added bus time for Route A: 2026-02-18 10:57:24.367227
Added bus time for Route B: 2026-02-18 10:39:24.367227
Next Route A bus: 2026-02-18 10:42:24.367227
Route A schedule: [datetime.datetime(2026, 2, 18, 10, 42, 24, 367227), datetime.datetime(2026, 2, 18, 10, 57, 24, 367227)]
Order added: 0001 (Student)
Order added: 0002 (Faculty)
Order added: 0003 (Student)
Order added: 0004 (Faculty)
Pending cafeteria orders: 4
Serving order: 0002 (Faculty)
Serving order: 0004 (Faculty)
Pending orders after serving: 2

Demo complete.
PS C:\AI Assistant Coding> 
```

Explanation:

Library Book Borrowing using a queue:

- The queue stores student names who request a book.
- When a student requests a book, we use `enqueue()` to add them to the queue.
- When a book becomes available, we use `dequeue()` to give it to the first student in line.
- This ensures fairness because the first requester gets the book first.

